คัมภีร์สองทิศ (twin_scroll)



ในหอคัมภีร์ หลินพบม้วนคัมภีร์โบราณที่อักษรขาดหายและมีตัวผิดปะปน ม้วนนี้ร้อยตัวอักษรด้วยสายสองทาง ไปหน้าและย้อนหลังได้ หลินถือพู่กันอยู่ระหว่างตัวอักษรสองตัว (ตำแหน่งเขียน) อรุณช่วยเลื่อนพู่กันซ้ายขวา เติมอักษรใหม่ และลบอักษรผิดที่อยู่ทางซ้ายของพู่กัน เพื่อซ่อมคาถาปลดผนึกประตูวังวน

สิ่งที่ฝึกในด่านนี้: ลิสต์โยงสองทางวงกลมที่มี `sentinel` + `cursor`: แทรก/ลบ/เลื่อนใน `O(1)`

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `twin_scroll.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

```
struct Rune {
    char data;
    Rune* prev;
    Rune* next;
};
Rune* makeScroll();
void insertLeft(Rune* sentinel, Rune*& cursor, char c);
void moveLeft  (Rune* sentinel, Rune*& cursor);
void moveRight (Rune* sentinel, Rune*& cursor);
void eraseLeft (Rune* sentinel, Rune*& cursor);
```

- `makeScroll()`: สร้างม้วนว่าง (`sentinel` หนึ่งโหนด ชี้ตัวเองทั้งสองทาง) แล้วคืนพอยน์เตอร์ของ `sentinel`
- `insertLeft(sentinel, cursor, c)`: แทรกตัวอักษร `c` ไว้ทางซ้ายของตำแหน่งเขียน (ก่อนโหนด `cursor`) ตำแหน่งเขียนยังอยู่ทางขวาของตัวอักษรที่เพิ่งแทรก `cursor` ไม่เปลี่ยน
- `moveLeft(sentinel, cursor)`: เลื่อนตำแหน่งเขียนไปซ้าย 1 ตัวอักษร ถ้าอยู่หน้าสุดแล้วไม่ต้องทำอะไร
- `moveRight(sentinel, cursor)`: เลื่อนตำแหน่งเขียนไปขวา 1 ตัวอักษร ถ้าอยู่ท้ายสุดแล้วไม่ต้องทำอะไร
- `eraseLeft(sentinel, cursor)`: ลบและ `delete` ตัวอักษรทางซ้ายของตำแหน่งเขียน ถ้าไม่มีตัวอักษรทางซ้ายไม่ต้องทำอะไร

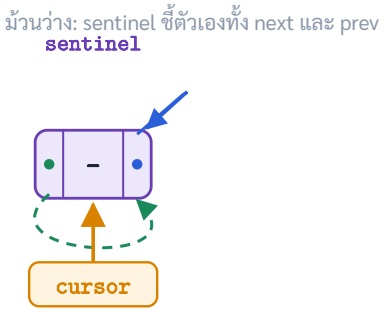
ม้วนคัมภีร์ใช้ `sentinel` เป็นโหนดพิเศษ (ไม่ใช่ตัวอักษร) ต่อกันเป็นวงกลมกับตัวอักษรทั้งหมด: `sentinel->next` คือตัวอักษรแรก และ `sentinel->prev` คือตัวอักษรสุดท้าย ม้วนว่างคือ `sentinel` ที่ชี้ตัวเองทั้ง `prev` และ `next` ทำให้ไม่มี `nullptr` และไม่มีกรณีพิเศษของหัวและท้าย

`cursor` คือโหนดที่อยู่ "ทางขวาของตำแหน่งเขียน" ถ้าตำแหน่งเขียนอยู่ท้ายสุด `cursor == sentinel` ดังนั้นตัวอักษรทางซ้ายของตำแหน่งเขียนคือ `cursor->prev` (ถ้ามันไม่ใช่ `sentinel`)

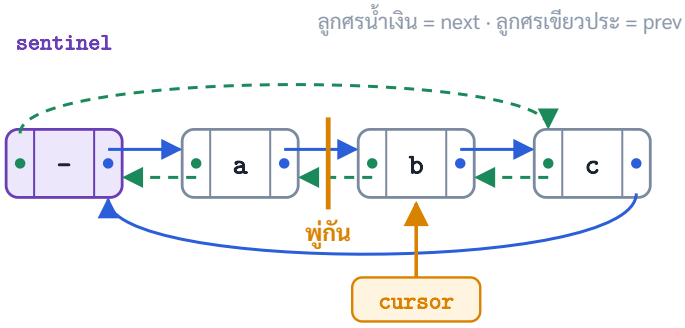
- `makeScroll` ต้องสร้าง `sentinel` ด้วย `new` ให้ `prev` และ `next` ชี้ตัวเอง
- หลังทุกคำสั่ง โปรแกรมตรวจเช็คว่า `prev`/`next` ของ `cursor` และเพื่อนบ้านชี้ไปมาถูกต้อง และเมื่อจบจะเดินทั้งม้วนตรวจลูกศรทั้งสองทิศและตำแหน่งของ `cursor`

- `insertLeft` ต้องสร้างโหนดใหม่หนึ่งโหนดพอดี · `moveLeft`, `moveRight`, `eraseLeft` ห้ามสร้างโหนด · `eraseLeft` ต้อง `delete` ตัวอักษรที่ลบ · ห้าม `delete sentinel`
- ทุกคำสั่งต้องใช้เวลา `O(1)` — ห้ามเดินม้วนจากหัวเพื่อหาโหนดก่อนหน้า (มี `prev` ให้แล้ว)

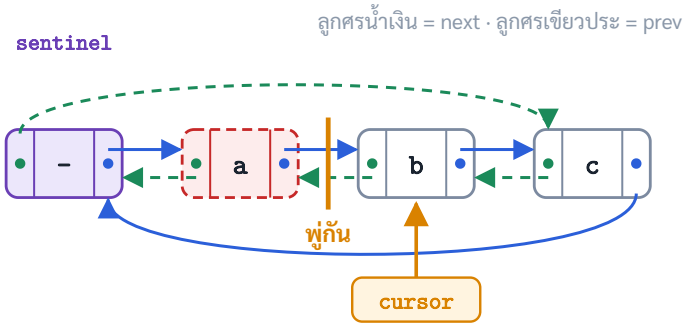
`makeScroll()`: ม้วนว่าง



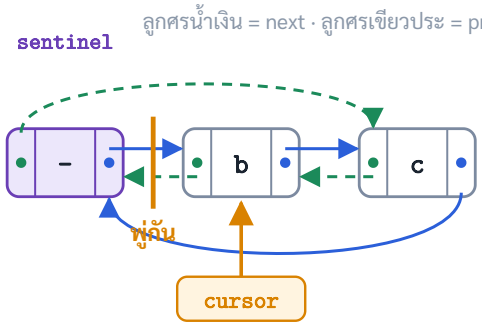
ข้อความ "abc" และตำแหน่งเขียนอยู่ระหว่าง a กับ b (`cursor` ชี้ b)



`eraseLeft`: โหนด a ที่อยู่ทางซ้ายของตำแหน่งเขียนจะถูกลบ



หลัง `eraseLeft`: ต่อ `prev`/`next` ข้ามโหนด a แล้ว `delete` ได้ "bc" ตำแหน่งเขียนอยู่หน้าสุด



รูปที่ 1 ลิสต์โยงสองทางวงกลมที่มี `sentinel` (ลูกศรน้ำเงินคือ `next` ลูกศรเขียวคือ `prev`)

การส่งงาน: ดาวน์โหลดไฟล์แนบ `twin_scroll.cpp` (ไฟล์โครง) และ `twin_scroll.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `twin_scroll.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `twin_scroll.cpp` ห้ามแก้ไขไฟล์ `twin_scroll.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp twin_scroll.cpp -o twin_scroll`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียก (เริ่มจากข้อความ "abc" ตำแหน่งเขียนอยู่ท้าย)	ข้อความ (เครื่องหมาย | คือตำแหน่งเขียน)
เริ่มต้น	abc|
`moveLeft`	ab|c
`moveLeft`	a|bc
`eraseLeft`	|bc
`eraseLeft` (อยู่หน้าสุดแล้ว)	|bc (ไม่เปลี่ยน)
`insertLeft('x')`	x|bc
`moveRight` สามครั้ง	xbc| (ครั้งที่สามไม่ทำอะไรเพราะอยู่ท้ายสุดแล้ว)

ตารางแสดงการเรียกต่อเนื่องกันตามลำดับ

ข้อจำกัด

- $1 \le Q \le 100$
- $0 \le N \le 200\ 000$ และ $0 \le M \le 200\ 000$ ในแต่ละชุด และผลรวมของ $N + M$ ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 400 000
- ข้อความเริ่มต้นและอักษรที่ใช้แทรกเป็นตัวพิมพ์เล็ก `a`–`z`
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (15 คะแนน) มีเฉพาะคำสั่ง `P`, $N \le 20$ และ $M \le 50$
2. (25 คะแนน) ไม่มีคำสั่ง `B`, $N \le 100$ และ $M \le 500$
3. (25 คะแนน) $N \le 100$ และ $M \le 500$
4. (35 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
แต่ละชุด: บรรทัดแรกมี N และ M ถ้า $N > 0$ บรรทัดถัดไปคือข้อความเริ่มต้นยาว N ตัวอักษร (ไม่มีช่องว่าง; ถ้า $N = 0$ จะไม่มีบรรทัดนี้) แล้วตามด้วยคำสั่ง M บรรทัด: `L` · `R` · `B` · `P c`

โปรแกรมตรวจเรียก `makeScroll` แล้วใช้ `insertLeft` สร้างข้อความเริ่มต้นทีละตัวอักษรโดยให้ตำแหน่งเขียนอยู่ท้ายสุด จากนั้นทำคำสั่งตามลำดับ: `L` $=$ `moveLeft` $\cdot$ `R` $=$ `moveRight` $\cdot$ `B` $=$ `eraseLeft` $\cdot$ `P c` $=$ `insertLeft` ตัวอักษร c

พิมพ์ 1 บรรทัดต่อหนึ่งชุด: ความยาวของข้อความ จำนวนตัวอักษรที่อยู่ทางซ้ายของตำแหน่งเขียน และข้อความสุดท้าย (พิมพ์ - ถ้าข้อความว่าง) คั่นด้วยช่องว่าง

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
1 3 4 abc L L B P x	3 1 xbc

abc| $\to$ ab|c $\to$ a|bc $\to$ ลบ a ได้ |bc $\to$ แทรก x ได้ x|bc จึงมีข้อความ xbc ยาว 3 ตัว และมีตัวอักษรทางซ้ายพู่กัน 1 ตัว
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 3, 4

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
2 0 3 P a P b P c 1 0 q	3 3 abc 1 1 q

มีเฉพาะคำสั่ง `P` ตำแหน่งเขียนอยู่ท้ายสุดตลอด · ชุดที่สองไม่มีคำสั่ง ผลคือข้อความ q กับพู่กันอยู่ท้ายสุด
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3, 4